

2. Introduction to Dynamic Memory Allocation. DMA functions malloc(), calloc(), free() etc.

◆ 1. What is Memory Allocation?

Memory allocation is the process of **reserving memory space in RAM** for storing data.

There are **two types**:

- **Static Memory Allocation** - Fixed room booked in advance
 - **Dynamic Memory Allocation** - Book room when you reach hotel
-

◆ 2. Static vs Dynamic Memory Allocation

☑ Static Memory Allocation

- Memory allocated **at compile time**
- Fixed size
- Cannot change during execution

📌 Example:

```
int a[10];
```

✗ Problem: Wastage or shortage of memory

☑ Dynamic Memory Allocation (DMA)

📌 Definition

Dynamic Memory Allocation is the process of allocating memory **at run time** using standard library functions.

✓ Why DMA is Needed?

- When memory size is **not known in advance**
 - Efficient memory usage
 - Required for **linked list, stack, queue, tree**
-

◆ 3. Header File for DMA

```
#include <stdlib.h>
```

◆ 4. DMA Functions in C

Function	Purpose
<code>malloc()</code>	Allocates single block of memory
<code>calloc()</code>	Allocates multiple blocks
<code>realloc()</code>	Resizes allocated memory
<code>free()</code>	Releases memory

◆ 5. malloc() Function

✓ Definition

`malloc()` allocates **single block of memory** at run time.

✂ Memory contains **garbage values**.

✓ Syntax

```
ptr = (datatype *) malloc(size);
```

✓ Example

```
int *p;  
p = (int *) malloc(sizeof(int));
```

✓ Program Example

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p;
    p = (int *) malloc(sizeof(int));

    *p = 10;
    printf("%d", *p);

    free(p);
    return 0;
}
```

✓ Advantages

- Efficient memory usage
 - Flexible size
-

✗ Disadvantages

- Garbage values
 - Programmer must free memory
-

◆ 6. calloc() Function

✓ Definition

`calloc()` allocates **multiple blocks of memory** and **initializes all values to zero**.

✓ Syntax

```
ptr = (datatype *) calloc(n, size);
```

✓ Example

```
int *p;
p = (int *) calloc(5, sizeof(int));
```

✓ Program Example

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p;
    p = (int *) calloc(5, sizeof(int));

    for(int i = 0; i < 5; i++)
        printf("%d ", p[i]);

    free(p);
    return 0;
}
```

✓ Output

```
0 0 0 0 0
```

◆ 7. Difference Between malloc() and calloc()

malloc()	calloc()
Allocates single block	Allocates multiple blocks
Garbage values	Initializes to zero
One argument	Two arguments
Faster	Slightly slower

◆ 8. realloc() Function

✓ Definition

`realloc()` changes the **size of previously allocated memory**.

✓ Syntax

```
ptr = (datatype *) realloc(ptr, new_size);
```

✓ Example

```
p = (int *) realloc(p, 10 * sizeof(int));
```

✓ Use

- Increase or decrease memory size
-

◆ 9. free() Function

✓ Definition

`free()` releases the allocated memory.

📌 Prevents **memory leakage**.

✓ Syntax

```
free(ptr);
```

✓ Example

```
free(p);  
p = NULL;
```

◆ 10. Memory Leak

📌 Definition:

Memory leak occurs when allocated memory is **not freed** after use.

✗ Example

```
p = (int *) malloc(sizeof(int));  
// free(p) missing
```

✓ Solution

Always use:

```
free(p);
```

◆ 13. Short Exam Questions

Q: Which function initializes memory to zero?

👉 `calloc()`

Q: Which function releases memory?

👉 `free()`
